

API preparation

Table of Contents

1.1 API Authentication

1.2 Status Codes

1.3 Human Handoff Webhook Events

API Authentication

MaiAgent uses API keys for authentication.

Authentication Method

All API requests must include an API Key in the HTTP headers, in the following format:

```
const headers = {  
  "Authorization": "Api-Key YOUR_API_KEY_HERE",  
  "Content-Type": "application/json"  
}
```

How Do I Obtain an API Key?

Log in to the MaiAgent admin console, then follow these steps:

Click the **Username** dropdown in the upper-right corner.

Click **Account** to open your profile page.

Switch to the **API Keys** page.

Click **Create Key**, enter a name, and set an expiration date if needed. Keys without an expiration date do not expire automatically.

Immediately copy the full key after creation and store it securely.

Security reminder: Keep your API key secure and do not share it publicly Key format: API keys contain a combination of letters and numbers

The full key is displayed only once, immediately after it is successfully created. Subsequent listings show only some characters, and you cannot view or recover the full key again. If you did not save it, create a new key, update the services that use it, and then revoke the old key.

Cannot Find an Old System-Issued Key

The **API Keys** page in your personal account lists only keys you created yourself. When you first join an organization (including your first login through AD), the system may automatically issue a key with no expiration date, but it does not provide the full value, so you cannot obtain that key to call the API. To call

the API, create a new key on your personal account page and save the full value immediately after successful creation.

For keys issued automatically by the system or on your behalf by an administrator, only members with organization key management permissions can view the owner, prefix, issuance source, status, and expiration date in [Organization Key Management](#). This page does not display the full value. If you do not have this permission, ask the organization owner or a member with organization key management permissions to help identify or revoke the old key.

API Key Permission Scope

When you call the API with an API key, its permissions match the key owner's role permissions in that organization. If the required permissions are missing, the API returns 403 Forbidden.

Write operations on members (members) and roles (groups) require the [Organization Overview Permission](#) subpermission under [Organization Permissions](#). Organization owners are exempt from this requirement. This applies to the following operations:

Add a member: `POST /api/v1/organizations/{organizationPk}/members/`

Remove a member: `DELETE /api/v1/organizations/{organizationPk}/members/{id}`

Change a member's roles: `POST /api/v1/organizations/{organizationPk}/members/{id}/update-groups/`

Create, update, or delete roles: `POST`

`PUT`

`PATCH`

`DELETE /api/v1/organizations/{organizationPk}/groups/...`

Add or remove role members in bulk, or assign or remove AI assistants, inboxes, knowledge bases, and databases for a role: `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-*`

`...`

Other write operations on members and roles, such as importing members in bulk, are subject to the same requirement

Permissions are evaluated at the subpermission level. An unchecked parent [Organization Permissions](#) option in role settings does not mean that its [Organization Overview Permission](#) subpermission is unchecked. To troubleshoot a 403 response, go to [Organization Settings → Roles](#) in the admin console, expand [Organization Permissions](#), and check whether [Organization Overview Permission](#) is selected for the key owner's role.

Error Handling

When authentication fails, the API returns the HTTP error code 401 Unauthorized:

Invalid or Missing API Key

```
{
  "detail": "Authentication credentials were not provided."
}
```

Suggested fix: Check that the request headers contain the correct Authorization field. The format should be: `Authorization: Api-Key YOUR_API_KEY`

Incorrect API Key Format

```
{
  "detail": "Invalid API key format."
}
```

Suggested fix: Verify the API key format. Make sure you use the correct prefix, `Api-Key` rather than `Bearer`, and that the key itself contains no extra spaces or characters.

Status Codes

When sending an HTTP request via the API, you can use the following status codes to check if the API request was successful:

Status Code Descriptions

Success Codes

Status Code	Description
200	Request successful
201	Resource created successfully

Success Response Example

This is a content demonstration. The response format may vary for different endpoints. Please refer to the documentation for the specific endpoint.

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // Result content returned
  ]
}
```

Error Codes

Status Code	Description
400	Bad Request
401	Authentication failed
403	Permission denied

Status Code	Description
404	Resource not found
500	Server error

Error Response Example

400 - Bad Request: Required field missing

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

Human Handoff Webhook Events

At each stage of the human handoff process—when AI determines that a human is needed, an agent is assigned, the conversation returns to AI, or the queue times out—MaiAgent sends an event to your configured endpoint via HTTPS POST. This lets dispatching, alerts, CRM, or your own customer service system take over automatically without polling. Event names describe handoff state transitions and form a fixed public contract. Each event has a corresponding `event_type` in the conversation event log API (`GET /api/v1/conversations/{id}/events/`), as shown below, for reconciliation or retrieving the same information through polling.

Enabling Webhooks

Customer administrators or integration staff can configure this themselves in the admin console without engineering assistance:

Admin console → Settings → Inboxes → Select an inbox → “Human Handoff” tab → “Notifications” subtab

Enable “Webhook Notifications (System Integration)” and enter the receiver URL (only URLs starting with `https://` are accepted)

(Optional but recommended) Enter a signing secret to enable source verification (see “Signature Verification” below)

Webhook, Notification Center, and Email each have an independent toggle and do not affect one another. Webhooks are disabled by default and enabled per inbox. This channel is separate from the inbox's existing “Message Webhook” for sending and receiving messages.

Event Types

event	Event log API <code>event_type</code>	When sent	Notes
<code>handoff.requested</code>	<code>transfer_triggered</code>	AI determines a human is needed: the conversation enters the queue and AI stops replying automatically	The first event in each handoff process; <code>assignee</code> is <code>null</code> . If human agents work in your own system, this is the signal to take over
<code>handoff.assigned</code>	<code>human_assigned</code>	The conversation is assigned to a customer service member in the MaiAgent admin console	<code>assignee</code> is the assigned agent; transferring to another

event	Event log API <code>event_type</code>	When sent	Notes
		(including manual assignment, self-assignment, transfer, and automatic assignment)	agent sends another event with a different event <code>id</code>
<code>handoff.returned_to_ai</code>	<code>returned_to_ai</code>	An agent clicks “Return to AI Agent” in the admin console or returns the conversation to AI through the <code>return-to-ai</code> API	<code>assignee</code> is <code>null</code> (the conversation no longer has an assigned human agent)
<code>handoff.queue_timeout</code>	<code>queue_timeout</code>	The queue exceeds the inbox's configured “Wait Timeout” without anyone taking over, and MaiAgent automatically returns the conversation to AI	<code>assignee</code> is <code>null</code> . If human agents work in your own system, this means MaiAgent has resumed AI replies without your knowledge. Treat it as an exception that requires an alert, or set the wait timeout to 0

All four event types have identical payload keys, signatures, and retry behavior. The receiver should **route events by the `event` field**. New event types may be added in the future. Return 2xx and ignore any unrecognized `event` rather than treating it as an error.

Both `handoff.returned_to_ai` and `handoff.queue_timeout` mean that AI has resumed automatic replies. The former is initiated by a person (an agent or your system), while the latter happens automatically on timeout. Neither means that the conversation is closed. No event is sent when an agent directly marks a conversation as “Resolved” or when a conversation re-enters the queue because its agent loses inbox access. For a complete resolution signal, query `status` through the conversation API or observe changes to `conversation.progress_status` in the Message Webhook.

Event Format

```
POST
Content-Type: application/json
```

```
{
  "event": "handoff.assigned",
  "id": "9f0e1d2c- ... ",
  "occurred_at": "2026-09-09T02:30:00.123456+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "url": "https://admin.maiagent.ai/ ... "
  },
  "inbox": { "id": "0f9a3d42- ... " },
  "organization": { "id": "b1d20c77- ... " },
  "contact": { "id": "a4c1f6b3- ... " },
  "assignee": { "id": "3e8f5a90- ... " }
}
```

Field	Type	Description
event	string	Event name (see “Event Types” above)
id	string (uuid)	Event identifier, matching the <code>id</code> returned by the event log API. Retries send exactly the same content; use it for deduplication
occurred_at	string (date-time)	Time the event occurred (ISO 8601). Delivery order is not guaranteed; use it for ordering
conversation.id	string (uuid)	Conversation identifier
conversation.url	string (uri)	Link to the conversation page in the admin console
inbox.id	string (uuid)	Inbox identifier
organization.id	string (uuid)	Organization identifier
contact	object	null
assignee	object	null

Field names and nesting match the MaiAgent standard format for the inbox's “Message Webhook” (snake_case, with related objects nested as `{ "id" }`), so the same receiver can handle both sets of Webhooks. The payload contains only identifiers, the event time, and an admin console link. It **does not contain conversation text or the reason for handoff**. To retrieve conversation content, use the identifiers to query the conversation and message APIs.

Signature Verification (Recommended)

After you configure a signing secret in the admin console, each delivery includes the following header:

```
X-MaiAgent-Signature-256: sha256=
```

The value is a hexadecimal HMAC-SHA256 digest computed with the secret over the **raw request body bytes**. Always verify against the raw bytes you received. Do not parse and then reserialize the body, as differences in field order or whitespace will produce a different digest.

```
import hashlib
import hmac

def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
    hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

The secret is write-only. After saving, it is not displayed again; the admin console shows only “Configured”. Saving with the field empty keeps the existing secret. Enter a new value to replace it, or use “Clear Secret” to remove it. If no secret is configured, deliveries do not include the signature header.

Minimal Receiver Example

The following Python (Flask) receiver can be started directly. It reads raw bytes for signature verification, routes by `event`, and returns 2xx before doing time-consuming processing. Set `HANDOFF_WEBHOOK_SECRET` to the signing secret configured in the admin console.

```

import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]

def verify_signature(raw_body: bytes, header_value: str | None) → bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)

@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data() # Raw bytes; do not reserialize request.json
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    if already_processed(event["id"]): # Retries send identical content;
deduplicate by event id
        return "", 204

    conversation_id = event["conversation"]["id"]
    if event["event"] == "handoff.requested":
        # AI has stopped replying; if human agents work in your system, open a
ticket and assign an agent here
        open_ticket(conversation_id, event["conversation"]["url"])
    elif event["event"] == "handoff.assigned":
        # Assigned to an agent in MaiAgent; another event arrives on transfer,
with a different assignee
        assign_ticket(conversation_id, event["assignee"]["id"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(conversation_id)
    elif event["event"] == "handoff.queue_timeout":
        # MaiAgent has automatically resumed AI replies; alert if your agent is
still handling the conversation
        close_ticket(conversation_id, reason="queue_timeout")
    # Unknown event: return 2xx and ignore it instead of treating it as an error

```

```

    return "", 204 # Return 2xx within 10 seconds; move time-consuming work to
the background

def already_processed(event_id):
    ...

def open_ticket(conversation_id, url):
    ...

def assign_ticket(conversation_id, assignee_id):
    ...

def close_ticket(conversation_id, reason=None):
    ...

```

```
HANDOFF_WEBHOOK_SECRET=your_secret flask --app receiver run --port 8080
```

For local testing, use the following command to simulate a delivery (compute the signature with the same secret):

```

BODY='{ "assignee": { "id": "3e8f5a90-0000-0000-0000-000000000004" }, "contact":
{ "id": "a4c1f6b3-0000-0000-0000-000000000005" }, "conversation": { "id": "6c2b7c1e-
0000-0000-0000-
000000000001", "url": "https://admin.maiagent.ai/conversations/index?
conversationId=6c2b7c1e-0000-0000-0000-
000000000001" }, "event": "handoff.assigned", "id": "9f0e1d2c-0000-0000-0000-
000000000006", "inbox": { "id": "0f9a3d42-0000-0000-0000-
000000000002" }, "occurred_at": "2026-09-09T02:30:00.123456+00:00", "organization":
{ "id": "b1d20c77-0000-0000-0000-000000000003" } }'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "your_secret" | awk
'{print $NF}')
curl -X POST http://localhost:8080/webhooks/handoff
-H "Content-Type: application/json"
-H "X-MaiAgent-Signature-256: sha256=$SIG"
-d "$BODY"

```

Responses and Retries

The receiver should respond with a 2xx status code within 10 seconds. The response body can contain anything.

On timeout, connection failure, or a non-2xx response, the system automatically retries up to 3 times at 60-second intervals. Retries send exactly the same content as the original delivery (with the same `id`); deduplicate by event `id`. Events are sent as separate background tasks, so `handoff.requested` and the immediately following `handoff.assigned` may arrive out of order. Order them by `occurred_at`.

Webhook delivery failures do not affect the main handoff process, Notification Center notifications, or Email notifications.

Taking Over Conversations in an External System After Handoff

If your human agents reply in your own customer service system rather than the MaiAgent admin console, combine the events on this page with the inbox's existing “Message Webhook” and message API to synchronize conversations in both directions during handoff. All steps use existing capabilities and require no additional enablement.

If visitor channels (web, LINE, etc.) are already connected to your system and you use MaiAgent only as an AI engine, see the technical documentation on [Third-Party Customer Service System Integration \(Including Human Handoff\)](#) instead. This section covers visitors using MaiAgent channels while only the human agents work in your system.

Integration Flow

Detect the start of handoff: On receiving `handoff.requested`, open a ticket or create a session in your system using `conversation.id`, and record `contact.id`. A subsequent `handoff.assigned` includes `assignee.id`, which you can use to update the responsible agent. Set the inbox's “Wait Timeout” to 0 as recommended; otherwise, a timeout triggers `handoff.queue_timeout` and AI automatically resumes replies.

Receive customer messages and agent replies: In the inbox's “Message Webhook” settings in the admin console, add Webhooks for `incoming` (customer messages) and `outgoing` (AI or agent replies). Each message's `conversation` object contains the current processing state. Process only messages whose `progress_status` is `human_serving` to skip traffic in AI mode:

Field	Description
conversation.status	<code>open</code> <code>queued</code> <code>resolved</code>
conversation.progress_status	<code>ai_processing</code> <code>waiting_for_human</code> <code>human_serving</code>

Field	Description
	resolved open
conversation.auto_reply_enabled	Whether AI replies automatically; <code>false</code> after handoff
conversation.assignee	Responsible agent <code>{ "id", "name" }</code> ; <code>null</code> if no agent is responsible
sender.type	contact (customer) user (human agent) chatbot (AI)

Write agent replies back to the conversation: Call `POST /api/v1/messages/outgoing/` with an API key, including `conversation` and `content`. MaiAgent forwards the message to the customer's channel. The sender is the member who owns the API key, and that member must have access to this inbox (see the “Conversations and Messages” endpoint documentation).

End the handoff: When the agent finishes, call `PATCH /api/v1/conversations/{id}/return-to-ai/` so AI takes over again. Whether you call the API or an agent clicks “Return to AI Agent” in the MaiAgent admin console, you receive `handoff.returned_to_ai`. Use it to close the session in your system.

API Call Examples

(1) Message Webhook received during handoff (excerpt in `maiagent_standard` format)—use `conversation.progress_status` to determine whether the conversation is in human mode and `sender.type` to identify the speaker:

```
{
  "id": "9d2f7a10- ... ",
  "type": "incoming",
  "content": "I want to change my order address",
  "created_at": "2026-09-09T02:30:00+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "title": " ... ",
    "type": "individual",
    "status": "open",
    "progress_status": "human_serving",
    "auto_reply_enabled": false,
    "assignee": { "id": "3e8f5a90- ... ", "name": "Agent Xiaomei" }
  },
  "inbox": { "id": "0f9a3d42- ... ", "name": "Website Support", "channel_type":
"web" },
  "contact": { "id": "a4c1f6b3- ... ", "name": "Wang Xiaoming", "email": null,
"phone_number": null },
  "sender": { "id": "a4c1f6b3- ... ", "name": "Wang Xiaoming", "type": "contact" },
  "attachments": [],
  "metadata": {}
}
```

(2) Write an agent reply back to the conversation—only `conversation` and `content` are required; other fields are optional. The message is sent to the customer's channel as the member who owns the API key:

```
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"
-H "Authorization: Api-Key YOUR_API_KEY"
-H "Content-Type: application/json"
-d '{
  "conversation": "6c2b7c1e- ... ",
  "content": "Hello, your address has been updated to Xinyi District, Taipei..."
}'
```

(3) Return the conversation to AI after handling it—no request body is required. Success returns 200 and the conversation object; AI then resumes automatic replies and `assignee` is cleared:

```
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/6c2b7c1e- ... /return-
to-ai/"
-H "Authorization: Api-Key YOUR_API_KEY"
```

All three steps together (Python, `requests`):

```
import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": "Api-Key YOUR_API_KEY"}

def on_message_webhook(payload: dict) → None:
    conv = payload["conversation"]
    if conv["progress_status"] ≠ "human_serving":
        return # Do not route AI-mode traffic to the customer service system
    if payload["sender"]["type"] = "contact":
        push_to_agent_console(conv["id"], payload["content"]) # Customer message
        → your agent interface

def reply(conversation_id: str, text: str) → None:
    requests.post(f"{API}/messages/outgoing/", headers=HEADERS,
                  json={"conversation": conversation_id, "content": text},
                  timeout=10).raise_for_status()

def finish(conversation_id: str) → None:
    requests.patch(f"{API}/conversations/{conversation_id}/return-to-ai/",
                  headers=HEADERS, timeout=10).raise_for_status()
```

Notes

Replies written through the API are also sent through the `outgoing` Message Webhook (`sender.type` is `user`). Deduplicate by message `id` to avoid displaying them twice in your system.

Transferring to another agent sends another `handoff.assigned` , with `assignee.id` identifying the new responsible agent. Update the existing session by `conversation.id` instead of opening a duplicate ticket.

Message Webhooks and human handoff Webhooks have separate settings and signature mechanisms. The former sends the configured secret unchanged in the `X-Webhook-Secret` header; the latter uses the HMAC-SHA256 signature described on this page.